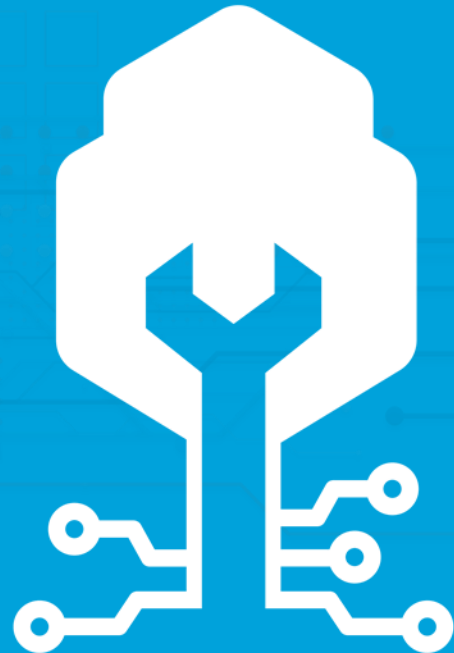


Databases

KEN2110

Denise Slenter

Adapted with gratitude from the original lectures by Dr. Ashish Sai, Dr. Katharina Schneider, Dr. Max Sondag, and dr. Tom Pepels



What have we looked at so far?

- Structure of data
 - Entities, Relationships, Attributes
- Database Systems
 - Databases:
 - Collection of logically related data, existing over a long period of time, managed by DBMS
 - Database Management System (DBMS):
 - Program used for defining and maintaining this database

Relational DBMS



Most DBMS are networked and multi-user.

This introduces a lot of extra complexity:

- Concurrent writes to files
- Network interruptions
- Database downtime while users are connected
- Requesting changing data
- Managing computational resources
- Authentication and authorization

MySQL is an example of a relational multi-user, networked DBMS

SQLite is an example of a single-user database without networking

DMBS rely on ACID

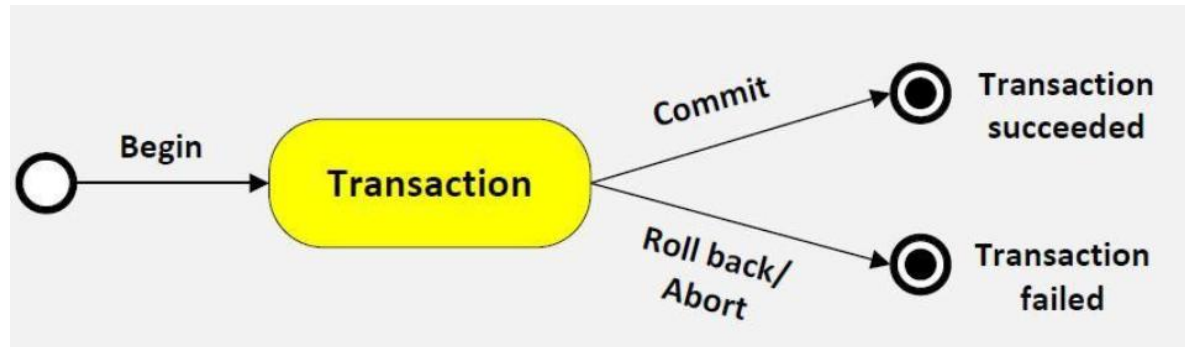
An important property of a multi-user DBMS is ACID:

Atomic: Transactions are all or nothing

Consistent: Only valid data is saved

Isolated: Simultaneous transactions are independently performed

Durable: Transactions are never lost



General concepts for relational data modeling



Some Relational Definitions

Entity:	Item of interest to the use of the database, e.g. Customer, Student, Part, Geolocation, etc.
Cell:	Individual piece of data stored in a table
Field:	Property of an entity (table column)
Row:	Complete instance of an item (record)
Result set:	Results of a query (subsection of the database)
Primary Key:	Unique identifier (ID) for each row in a table
Foreign Key:	Connect IDs from one table to another (one or more fields can be used)

Please note, terms in red are different from the SQL book p.8

Example Table (entity called Customer)



Question:

How many attributes does this entity describe?

ID	First Name	Last Name	Street Address	City	State
52	Denver	Emerson	856 Cook St.	Raleigh	NC
53	John	Emory	99 Hillsborough St.	Garner	NC
54	Ebony	Farmer	872 W. Morgan St.	Raleigh	NC
55	Kim	Doe	553 Wayne St.	Raleigh	NC
56	Coretta	Diaz	781 Bloodworth St.	Raleigh	NC
57	Victor	Denver	31 St. Mary's St.	Raleigh	NC
58	Hamish	David	21 Cameron Ct.	Raleigh	NC
59	Erin	Counts	763 E. West St.	Raleigh	NC
60	Harris	Collman	455 E. Edenton St.	Raleigh	NC
61	Melissa	Chantay	3201 Glenwood Ave.	Raleigh	NC
62	Emmanuel	Colin	11 Sassafras Way	Raleigh	NC
63	Illa	Carlson	563 Oberlin Rd.	Raleigh	NC
64	Tyrone	Rodgers	453 Pine St.	Raleigh	NC

Primary and Foreign keys

- Primary Key (PK)
 - A column (or set of columns) that uniquely identifies each row in a table
 - Example: StudentID in a Students table
 - Analogy: like your student number → unique to you, never duplicated
- Foreign Key (FK)
 - A column that references the primary key in another table
 - Creates the link between tables
 - Example: StudentID in an Enrollments table, pointing to Students.StudentID
 - Analogy: like writing your student number on an exam sheet → connects your work to you

👉 PK = identity. FK = connection.

Data Models



Data Models

- A database schema is written using a Data Definition Language (DDL)
 - In fact, in the DDL of a particular DBMS

```
CREATE TABLE organization (  
    organization_id INT PRIMARY KEY,  
    organization_name VARCHAR(150) NOT NULL UNIQUE,  
    organization_type VARCHAR(50) NOT NULL,  
    region_name VARCHAR(100) NOT NULL,  
    trust_level VARCHAR(30) DEFAULT NULL,  
    status VARCHAR(30) NOT NULL,  
    organization_notes VARCHAR(255) DEFAULT NULL  
);
```

- Too low level to describe the data requirements in an **understandable** way
- We require a higher-level description of the schema – Data Model

What is a Data Model?

- A data model is a notation for describing data or information
- The description consists of three parts:
 - Structure of the data
 - Operations on the data
 - Constraints on the data

What is a Data Model?

- A data model is a notation for describing data or information
- The description consists of three parts:
 - **Structure of the data**
 - Operations on the data
 - Constraints on the data

Data structures

- Model provides: What objects exist and how do they relate?
- Problem: Different (database) stakeholders
 - Domain experts: think in real-world concepts
 - Designers: think in logical schemas
 - DBMS: thinks in pages, indexes, bytes
- Thus, we need different levels of abstraction
 - Physical data model
 - (Logical data model)
 - Conceptual data model

Physical Data Models

- Define how data is actually stored, accessed and managed in a specific DBMS
- Examples:
 - Partitioning
 - Compression
 - Pointer structures
 - ...
- Used by Database admins and developers
- Answer: How can the DBMS store and access the data efficiently?

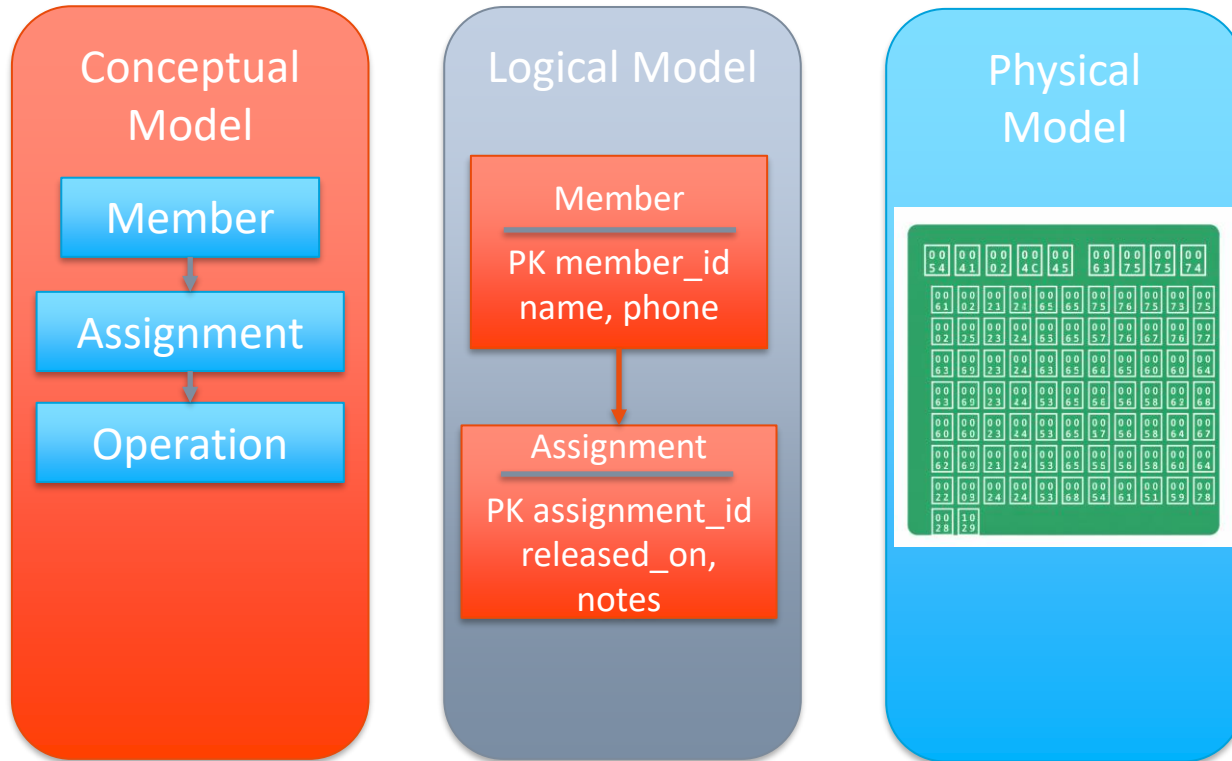
Logical Data Models

- Defines how the data is represented inside a database paradigm, independently of physical storage.
- Answer: How should the information be organized in the database?
- Examples:
 - Relational model
 - Graph model
 - Network model
 - ...

Conceptual Data Models

- To describe the structure of the data in terms of the real world
- Focusing on meaning rather than technical aspects
- Used by Business stakeholders, analysts, designers
- Answer: What information must the system remember?

Conceptual – Logical – Physical



What is a Data Model?

- A data model is a notation for describing data or information
- The description consists of three parts:
 - Structure of the data
 - Operations on the data
 - Constraints on the data

Operations on the data

- In programming: operations can be **anything** that can be programmed (e.g. program to replace every 10th occurrence of 'a' with 'z')
- In database data models: **operations limited** to a set of queries (operations that retrieve information) and modifications (operations that change the database)

What is a Data Model?

- A data model is a notation for describing data or information
- The description consists of three parts:
 - Structure of the data
 - Operations on the data
 - Constraints on the data

Constraints on the data

- A way of describing limitations on data by the database data model (e.g. a person cannot have a '-2' as an age)

Different Types of Data Models

- Conceptual
 - Entity-relationship
 - UML class diagrams
- Logical
 - **Relational**
 - Hierarchical
 - Network
 - Graph
 - Key-value
 - Object-based
 - Document-based
 - XML
 - JSON
- Physical
 - Index structures
 - File organization
 - Storage Layout

Relational Model



Why Relations?

- Very simple model
- Often matches how we think about data
- Relational Model is the data model that underlies SQL, the most popular database language today
- Relations (plus the operations with relations) form an algebra (relational algebra – well studied by database theorists)

Implementing Relations between entities

- One to One
 - Each row in Table A relates to at most one row in Table B
 - Users(UserID PK, Name, Email)
 - UserProfiles(UserID PK/FK → Users.UserID, Address, Phone)
 - Used when splitting data for security or optional info



Rare in practice, but useful for separating concerns (e.g. anonymized patient data)

Implementing Relations between entities

- One to Many
 - One row in Table A relates to many rows in Table B
 - Customer(CustomerID PK, Name)
 - Books(BookID PK, Title, AuthorID FK → Authors.AuthorID)
 - One Customer can order many books

👉 The most common type of relationship.

Implementing Relations between entities

- Many to Many
 - A row in Table A can relate to many rows in Table B, and vice versa
 - Authors(AuthorID PK, Name)
 - Books(BookID PK, Title, AuthorID FK → Authors.AuthorID)
 - BookAuthors (BookID FK, AuthorID FK)
 - A book can have many authors, and an author can write many books
- 👉 Many-to-many = always needs an extra table (aka junction or associative or linking or bridge table)

Relational Model

- Data is represented as a two-dimensional table called a **relation**

Title	Year	Length	Genre
Batman Begins	2005	220	Fantasy
Gone with the wind	1939	231	Drama
Star Wars	1977	124	sciFi

Movies

Relational Model

- Attributes – Columns: Title, Year, Length, Genre
- Tuples – Rows: Batman begins, Star wars, ...
- Relation Name - Movie

Title	Year	Length	Genre
Batman Begins	2005	220	Fantasy
Gone with the wind	1939	231	Drama
Star Wars	1977	124	sciFi

Movies

Relation Schema

- Name of a relation and the set of attributes

Movies(title, year, length, genre)

OR

Movies(title:String, year:integer, length:integer, genre:String)

Database Schema

- Relational Model: a database consist of one or more relations
- Set of relation schemas (each with distinct name) is called a relational database schema (or just a database schema)
- Database = collection of relations

Relational Model Constraints

- Conditions which must be present for a **valid relation**.
- Constraints are derived from the rules of the **domain** that the database represents (e.g. first picture movie in 1888 [1])
- There are different types of integrity constraints

Title	Year	Length	Genre
Batman Begins	2005	220	Fantasy
Gone with the wind	1939	231	Drama
Star Wars	1977	124	sciFi

Movies

Relational Model Constraints

- Conditions which must be present for a **valid relation**.
- Constraints are derived from the rules of the **domain** that the database represents (e.g. first picture movie in 1888 [1])
- There are different types of integrity constraints

Three main categories:

- Domain constraints
- Key Constraints
- Referential Integrity Constraints

Domain Constraints

- Can be violated if an attribute value is not appearing in the corresponding domain or it is not of the appropriate data type.
- Constraints on whether a cell can be NULL
- Constraints on the data types of records
- Constraints of the length / sign of data etc.

Table: students		
ID 	INT	NOT NULL
Last Name	VARCHAR(60)	NOT NULL
BirthDate	DATE	NOT NULL

Key Constraints

- An attribute that can uniquely identify a tuple in a relation is called the primary key of the table.
- The value of this attribute for different records in the relation has to be unique.
- CustomerID is a key attribute of Customer Table.

CustomerID 	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Referential Integrity Constraints

- Defines relations between entities
- Through using the PK in other tables as FK, relationships to data within tables are created.
- Data is depending on one another: removing or changing a row in one table, could destroy the meaning of other tables.
- Referential Integrity: logical dependency of FK on PK.

👉 DBMS throws an error and rejects an operation when a referential integrity constraint is broken

Referential Integrity Constraints

👉 DBMS throws an error and rejects an operation when a referential integrity constraint is broken

However, database behavior on deleting or updating records can be defined:

- Update/remove all linked records is possible (called CASCADE)

Entity Relationships



ER diagram (ERD) and model (ERM)

- ERD: visual representation of database structure
 - Relations are described, but for designing databases this has no consequence or meaning.
- ERM: blueprint of database

ER diagram (ERD) and model (ERM)

- ERD: visual representation of database structure
 - Relations are described, but for designing databases this has no consequence or meaning.
 - ERM: blueprint of database
- 👉 ERD = the idea. Schema = the build

ERD:

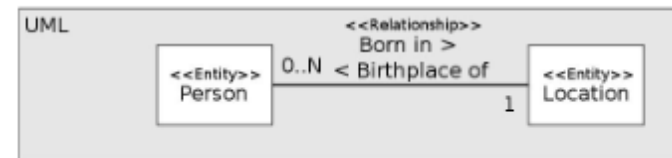
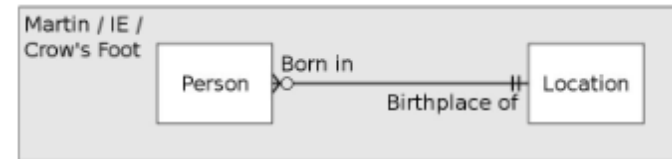
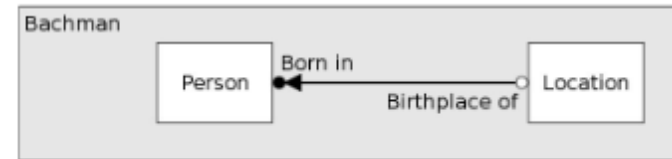
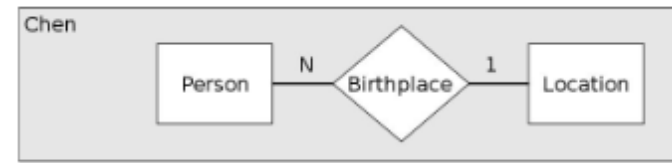
- Conceptual, high-level view
- Shows entities, attributes, and relationships
- Abstract and graphical → easy to understand

ERM (or schema):

- Concrete, technical view
- Defines tables, columns, datatypes, keys, constraints
- Implementation inside a specific DBMS

ER diagram (ERD) and model (ERM)

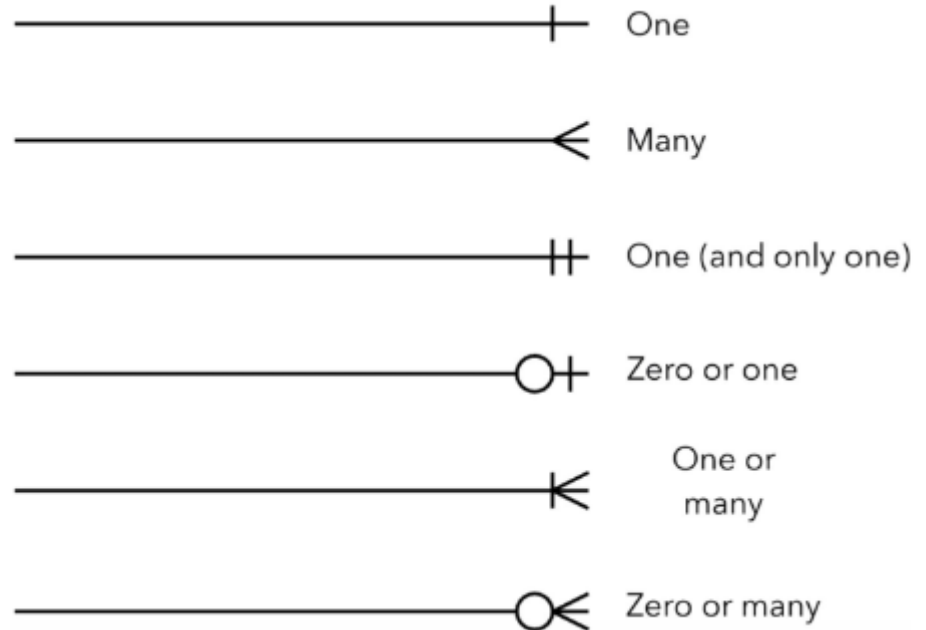
- ERD: visual representation of database structure
 - Relations are described, but for designing databases this has no consequence or meaning.
- ERM: blueprint of database



Crow's foot ERD

- Most commonly used notation for relations
- Three symbols:
 - Ring is zero
 - Dash is one
 - Crow's foot is many

Note: a regular arrowhead typically indicates the **direction of a FK reference** or points toward the **parent (referenced) table**, rather than defining record cardinality.



Crow's foot ERD

Let's try it out! Short background on problem:

A customer can buy products at a number of shops. When they make a purchase, we store their phone number, name and address. We keep track of sales made by vendors (who are not related to a shop). We want to be able to determine the customer, product, shop, and vendor for each sale.



Define the entities in this database, the attributes, and the relationships (don't worry at PK and FK for now). ~5 minutes

Short Break

- **10 minutes**



Crow's foot ERD

A customer can buy products at a number of shops. When they make a purchase, we store their phone number, name and address. We keep track of sales made by vendors (who are not related to a shop). We want to be able to determine the customer, product, shop, and vendor for each sale.

Entities:

- Customer
- Product
- Shop
- Sales
- Vendor

Crow's foot ERD

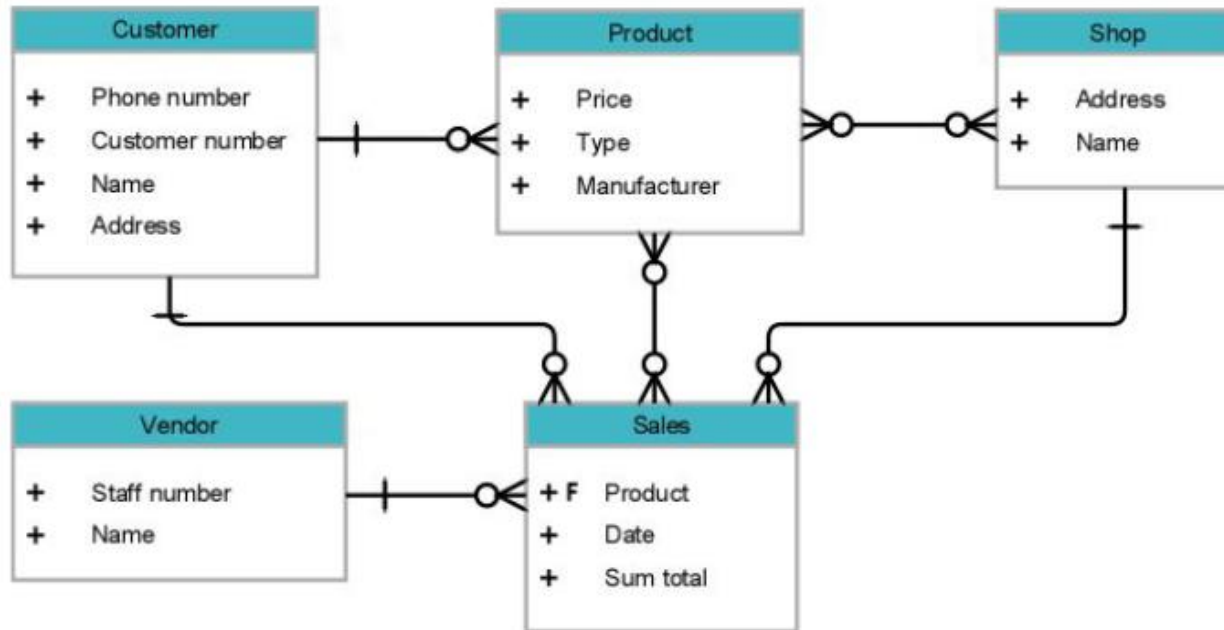
A customer can buy products at a number of shops. When they make a purchase, we store their phone number, name and address. We keep track of sales made by vendors (who are not related to a shop). We want to be able to determine the customer, product, shop, and vendor for each sale.

Entities && Attributes (note that some examples are not provided in background):

- Customer && Phone number, Name, Address
- Product && Price, In Stock
- Shop && name, address
- Sales && Product, Date, Shipment Timestamp
- Vendor && Staff number, name, rating for customer service

Crow's foot ERD

- Example ERD:



ERD as an ERM (or schema)

CREATE TABLE Customers(

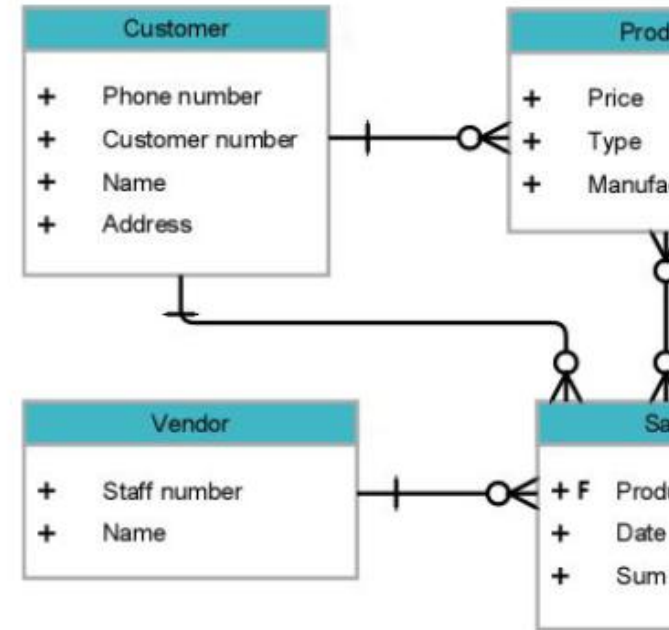
CustomerID **INT PRIMARY KEY**,

PhoneNumber **VARCHAR(15)**,

Name **VARCHAR(50)**,

Address **VARCHAR(100)**

);



ERD as an ERM (or schema)

CREATE TABLE Customers(

CustomerID **INT PRIMARY KEY**,

PhoneNumber **VARCHAR(15)**,

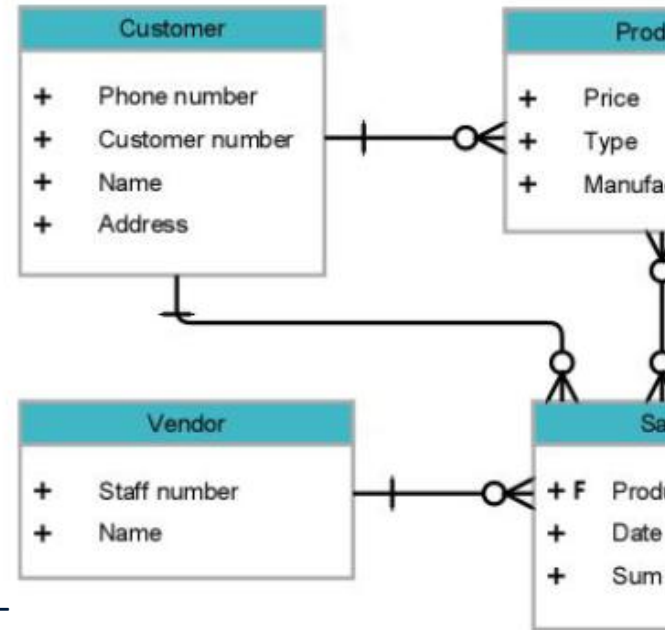
Name **VARCHAR(50)**,

Address **VARCHAR(100)**

);



Question: Do you think that modeling “Adress” as a ‘variable-length character type’ (varchar) with max. 100 characters provides enough structure to the underlying data?



Remember: Why Relations?

- Very simple model
- Often matches how we think about data
- Relational Model is the data model that underlies SQL, the most popular database language today
- Relations (plus the operations with relations) form an algebra (relational algebra – well studied by database theorists)

SQL (Part 1)



Pronunciation of SQL

- Option 1: "Ess-Queue-El" (S-Q-L)
- Option 2: "Sequel"
- Both are correct

General Information

- SQL is a very high-level language
 - Say “what to do rather” than “how to do it”
 - Avoid a lot of data-manipulation details needed in procedural languages like Java or C++
- DBMS figures out “best” way to execute query (query optimization)
- Latest version of the standard: ISO/IEC 9075-1:2023
<https://www.iso.org/standard/76583.html>

SQL as DDL



SQL as Data Definition Language (DDL)

- SQL is primarily a query language, for getting information from a database
- But SQL also includes a data-definition component for describing database schemas

SQL: Create



Creating (Declaring) a Relation

- Simplest form is:
- Most basic element:
 - Name of an attribute and its type

Example:

```
CREATE TABLE <name> (  
  <list of elements >  
);
```

- SQL names/identifiers must be:
 - No longer than 128 characters
 - Must start with a letter
 - Cannot contain spaces

```
CREATE TABLE Area (  
  area_id CHAR(4),  
  area_name VARCHAR (100),  
  status VARCHAR (30)  
);
```

ISO SQL Data Types

- Boolean: BOOLEAN
- Character:
 - CHAR(n) = fixed-length string of n characters
 - VARCHAR(n) = variable-length string of up to n characters
- Numeric:
 - NUMERIC, DECIMAL, INTEGER, SMALLINT, FLOAT, REAL, DOUBLE PRECISION
- Datetime:
 - DATE, TIME, TIMESTAMP

Java Type	Database Type	Nullability
boolean	BIT	No
java.lang.Boolean	BIT	Yes
byte	TINYINT	No
java.lang.Byte	TINYINT	Yes
double	DOUBLE	No
java.lang.Double	DOUBLE	Yes
float	REAL	No
java.lang.Float	REAL	Yes
int	INTEGER	No
java.lang.Integer	INTEGER	Yes
long	BIGINT	No
java.lang.Long	BIGINT	Yes
short	SMALLINT	No
java.lang.Short	SMALLINT	Yes
java.math.BigDecimal	DECIMAL	Yes
java.math.BigInteger	DECIMAL	Yes
char	CHAR	No
java.lang.Character	CHAR	Yes
java.lang.String	VARCHAR or CLOB	Yes
Serializable	BLOB	Yes
byte[]	BLOB	Yes
java.util.Date	TIMESTAMP	Yes
java.sql.Date	DATE	Yes
java.sql.Time	TIME	Yes
java.sql.Timestamp	TIMESTAMP	Yes

ISO SQL Data Types

- Standard data types: [SQL Data Types](#)
- Data types in various DBMSs:
 - http://www.w3schools.com/sql/sql_datatypes.asp

Check this page when working on the project implementation (starting next week with ERD!)

No need to memorize all these types for the exam, but the core SQL databases are required (next slide), including what these are useful for.

Core SQL datatypes

Numeric

- INT, BIGINT, SMALLINT → whole numbers
- DECIMAL(p,s) / NUMERIC(p,s) → exact numbers(good for money)
- FLOAT, REAL, DOUBLE → approximate values(scientific use)

Text

- CHAR(n) → fixed-length text
- VARCHAR(n) → variable-length text
- TEXT → long text, paragraphs

Date & Time

- DATE → year, month, day
- TIME → hours, minutes, seconds
- TIMESTAMP / DATETIME → date + time (optionally with time zone)

Binary / Special

- BLOB → large binary (images, audio, video)
- BOOLEAN → true/false values

Running Example: Limburg Mafia

```
CREATE TABLE Area (  
    territory_id CHAR(4),  
    territory_name VARCHAR (100),  
    status VARCHAR (30)  
);
```

Area

<u>territory_id</u>	territory_name	status
3008	'Bunde-Centrum'	'active'
3009	'Kerkrade-West'	'inactive'
3010	'Valkenburg-Heunsberg'	'active'
3011	'Sittard-Centrum'	'active'
3012	'Tegelen-Havenzone'	'inactive'

Running Example: Limburg Mafia

```
CREATE TABLE Financial_entry (  
    entry_id CHAR(4),  
    operation_id CHAR(4),  
    member_id CHAR(4),  
    organization_id CHAR(4),  
    entry_date DATE,  
    entry_type VARCHAR (50),  
    amount DECIMAL(12,2),  
    payment_method VARCHAR (30),  
    recorded_at TIMESTAMP  
);
```

Financial_entry

entry_id	operation_id	member_id	organization_id	entry_date	entry_type	amount	payment_method	recorded_at
8012	4006	1002	2003	'2025-01-20'	'closure_cost'	600.00	'cash'	'2025-01-20 18:42:15'
8013	4008	1007	NULL	'2024-05-10'	'collection'	350.00	'cash'	'2024-05-10 22:05:33'
8014	4010	1014	2008	'2024-11-15'	'expense'	520.00	'bank transfer'	'2024-11-16 09:12:44'
8015	4011	1003	NULL	'2024-03-01'	'cover_income'	780.00	'invoice'	'2024-03-02 10:25:11'

SQL Values

- Integers and reals (i.e. floating-point numbers) are represented as you would expect
- DECIMAL (p, s)
 - p = Precision → total number of digits (both sides of the decimal point)
 - s = Scale → number of digits after the decimal point
 - Example: DECIMAL(5,2) → up to 5 digits total, with 2 after the decimal. 123.45, 9.99
- Strings are too, except they require single quotes
 - Two single quotes = real quote, e.g. 'O''Brien'
- Any value can be NULL

SQL Values (continued)

- Dates and Times
 - DATE
 - form of a date value ``yyyy-mm-dd``
 - TIME
 - form of a time value ``hh:mm:ss``
 - TIMESTAMP
 - Date and time together ``yyyy-mm-dd hh:mm:ss``

SQL Integers

Data type	Range	Storage
BIGINT	-263 (-9,223,372,036,854,775,808) to 263-1 (9,223,372,036,854,775,807)	8 Bytes
INT	-231 (-2,147,483,648) to 231-1 (2,147,483,647)	4 Bytes
SMALLINT	-215 (-32,768) to 215-1 (32,767)	2 Bytes
TINYINT	0 to 255	1 Byte

SQL BLOBs

- Can store large data: documents, images, audio, video
- Space-efficient → storage allocated only when content is present
- Stores raw binary data (not interpreted as text or numbers)
- Great for unstructured or media-heavy content
 - And for keeping track of people we lend money to as Limburg Mafia 😄

SQL character strings

- CHAR(n) versus VARCHAR(n)
 - Fixed versus variable length
 - Uniform length values (e.g. 2 letter country codes) versus “free text”, e.g. names, book titles)
 - CHAR: Faster to compare and index
 - VARCHAR: Stores only what is needed

Declaring Keys

- An attribute or list of attributes may be declared **PRIMARY KEY** or **UNIQUE**
- Either says that no two tuples (i.e. data rows) of the relation may have the same values for the attribute(s) on the list.
- There are a few distinctions to be mentioned later

Declaring Single-Attribute Keys

- Place PRIMARY KEY or UNIQUE after the type in the declaration of the attribute
- Example:

```
CREATE TABLE Area (  
    area_id CHAR(4) PRIMARY KEY,  
    area_name VARCHAR (100),  
    status VARCHAR (30)  
);
```

Declaring Single-Attribute Keys (continued)

- Example of two alternative keys:

```
CREATE TABLE Member (  
    member_id CHAR(4) PRIMARY KEY,  
    alias VARCHAR(100),  
    first_name VARCHAR(100),  
    last_name VARCHAR(100),  
    member_role VARCHAR(50),  
    join_date DATE,  
    status VARCHAR(30),  
    phone VARCHAR(30) UNIQUE  
);
```

Declaring Multi-attribute Keys

- A key declaration can also be another element in the list of elements of a `CREATE TABLE` statement
- This form is essential if the key consists of more than one attribute.
 - May be used even for one-attribute keys.

Declaring Multiattribute Keys - Example

- We assume that for each model X purchased by customer Y there is one entry per day in table Sales.

Thus:

```
CREATE TABLE Assignment (  
    member_id    CHAR (4),  
    operation_id  CHAR (4),  
    assignment_role  VARCHAR(50),  
    assigned_on    DATE,  
    released_on    DATE,  
    assignment_status  VARCHAR(30),  
    PRIMARY KEY ( member_id, operation_id)  
);
```

PRIMARY KEY vs. UNIQUE

- There can be only one PRIMARY KEY for a relation, but several UNIQUE attributes
 - No attribute of a PRIMARY KEY can ever be **NULL** in any tuple.
 - But attributes declared UNIQUE may have NULL values, and there may be several tuples with NULL.

Altering and Deleting Relations

- To alter the schema of a relation:
 - Add a column
 - Drop a column
- To delete a relation

```
ALTER TABLE Member  
ADD date_of_birth DATE;
```

```
ALTER TABLE Member  
DROP COLUMN date_of_birth;
```

```
DROP TABLE Member;
```

Inserting Rows

- To insert a new row:

```
INSERT INTO table_name(column1, column2, column3, ...)  
VALUES (value_column1, value_column2, value_column3);
```

- Example:

```
INSERT INTO Area(area_id, area_name, status)  
VALUES (3007, 'Sittard-North', 'active');
```

Deleting Rows

- To delete a row:

```
DELETE FROM table_name  
WHERE condition;
```

- Example:

```
DELETE FROM Member  
WHERE member_id= '1005';
```

Update a tuple

- Can we update a value only in a tuple?

```
UPDATE table name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

- Be careful: Where clause is not necessary. But if you omit it, all records in the table will be updated

Example UPDATE

- Set the first_name of Bram in table Member to Braam

```
UPDATE Member  
SET first_name = 'Braam'  
WHERE first_name = 'Bram';
```

Updated the typo ;)



Question: Do you think that matching against any record where the first name is “Bram” is a good way to alter your database?

Overview of common SQL commands and their meaning

Command	Description	Example
SELECT	Retrieves data from a table	<code>SELECT * FROM table_name;</code>
INSERT INTO	Inserts new data into a table	<code>INSERT INTO table_name (col1, col2) VALUES (val1, val2);</code>
UPDATE	Modifies existing records	<code>UPDATE table_name SET col1 = val1 WHERE col2 = val2;</code>
DELETE	Removes records from a table	<code>DELETE FROM table_name WHERE condition;</code>
CREATE TABLE	Creates a new table	<code>CREATE TABLE table_name (col1 type1, col2 type2, ...);</code>
DROP TABLE	Deletes a table	<code>DROP TABLE table_name;</code>
ALTER TABLE	Modifies a table	<code>ALTER TABLE table_name ADD column_name column_type;</code>
JOIN	Combines rows from two or more tables	<code>SELECT * FROM table1 INNER JOIN table2 ON table1.col1 = table2.col2;</code>
GROUP BY	Groups rows by the values of one or more columns	<code>SELECT COUNT(*), col1 FROM table_name GROUP BY col1;</code>
ORDER BY	Sorts the result set	<code>SELECT * FROM table_name ORDER BY column_name ASC;</code>

SQL tips

- Capitalisation of SQL statements is a norm, but not mandatory.
- English-like syntax → easy to read and write
- Whitespace doesn't matter → use spaces/line breaks for readability
- Each statement ends with ; (semicolon)
- Capitalization: keywords not case-sensitive, but table/column names may be (depends on DB and quoting)
- Comments supported: -- single line or /* multi-line */
- Simple but powerful → combining statements unlocks real strength

SQL versus DBMS

- Non-procedural: you describe what you want, not how to get it
- Think of it as a communication language with the DBMS
- The DBMS decides how and when to execute your statement (query planner, optimizer)
- To use SQL inside software, you need an SDK/driver (e.g., JDBC, psycopg2, sqlite3) to talk to the DBMS

👉 SQL = Ask. DBMS = Do.

Questions?



Self study material

Data Types: Learning SQL, P.20 - P.27

- <https://dev.mysql.com/doc/refman/8.0/en/data-types.html>
- https://www.w3schools.com/sql/sql_datatypes.asp

CREATE / ALTER Table: Learning SQL, P.27 - P.33

- https://www.w3schools.com/sql/sql_create_table.asp
- https://www.w3schools.com/sql/sql_alter.asp
- <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>
- <https://dev.mysql.com/doc/refman/8.0/en/alter-table.html>

INSERT Learning SQL, P.37

- https://www.w3schools.com/sql/sql_insert.asp
- <https://dev.mysql.com/doc/refman/8.0/en/insert.html>

UPDATE Learning SQL, P.38 - P.39

- https://www.w3schools.com/sql/sql_update.asp
- <https://dev.mysql.com/doc/refman/8.0/en/update.html>

SELECT Learning SQL, Chapter 3

- https://www.w3schools.com/sql/sql_select.asp
- <https://dev.mysql.com/doc/refman/8.0/en/select.html>

DELETE Learning SQL, P.39

- https://www.w3schools.com/sql/sql_delete.asp
- <https://dev.mysql.com/doc/refman/8.0/en/delete.html>

DROP Learning SQL, P.43

- https://www.w3schools.com/sql/sql_drop_table.asp
- <https://dev.mysql.com/doc/refman/8.0/en/drop-table.html>

See you on Friday (tutorial!)

Please note that group projects will close today at 18.00

Groups with 1 or 2 students will be merged by the coordinator

No signup means no feedback on project (you're welcome to execute the tasks by yourself to help you study!)

